

SE4041 - Mobile Application Design and Development (MADD)

ASSIGNMENT 02

Proposal Report

StudyWell

Part A — Advanced iOS Application

QuizRoom

Part B — Innovative tvOS Prototype

Student ID: IT23176010

BSc (Hons) in Information Technology

Year 4 Semester 1

Document map

1. Assignment alignment and evidence checklist

2. Part A — StudyWell: purpose, design and implementation

3. Part A — user guide, testing and reflection

4. Part B — QuizRoom: concept, implementation and comparison

5. Part B — pitch, market and business strategy

6. Development workflow, AI disclosure and originality

7. Demonstration plan and final submission checklist

Appendices — screenshots, test evidence and AI interaction log

1. Assignment alignment and evidence checklist

This report follows the supplied SE4041 Assignment 02 brief (8 pages). Part A asks for a fully functional iOS app with at least three navigable screens, advanced SwiftUI UI, advanced persistence and meaningful integration of an Apple framework. Part B asks for a working visionOS/tvOS prototype and an investor/stakeholder pitch. The brief also requires GitHub repositories, README build instructions, AI-assistance documentation, and a viva demonstration.

Assessment area	Report evidence / outstanding action
Part A: purpose and audience	§2.1
Part A: UI, customization, navigation	§2.2–2.4; add screenshots and verify actual implementation
Part A: persistence and advanced framework	§2.5–2.6; confirm on-device results
Part A: testing, user guide, reflection	§3; insert actual test results
Part B: concept, similar apps, prototype	§4; complete external comparison
Part B: pitch and market strategy	§5; proposed model, not validated market data
Both: GitHub, AI use, originality, viva	§6–7 and appendices

2. Part A — StudyWell (iOS)

2.1 Purpose, problem and target audience

StudyWell is a student-focused study and wellness planner. It addresses the practical problem of managing academic deadlines, personal tasks and study-related routines while keeping daily movement goals visible. Its primary target users are university students who need a simple mobile workflow for organising tasks and monitoring healthy habits.

The central design decision is to combine a task planner with optional step-count information rather than treat productivity and wellness as unrelated activities. The app is intended for personal use, with an accessible, low-friction workflow.

2.2 Features and navigable screens

Today dashboard: overview of tasks, deadlines and progress.

Tasks: browse, search/filter, complete, reopen and delete saved tasks.

Add/Edit Task: capture title, module/category, priority, due date and notes.

Goals & Wellness: weekly completion goal and daily step target with available HealthKit step data.

Deadline reminders: local notifications when authorised by the user.

EVIDENCE TO ADD / VERIFY: Insert labelled screenshots for each actual screen and confirm navigation and feature names against the final Xcode build.

2.3 Advanced UI, accessibility and branding

The interface is implemented using SwiftUI views, reusable components and a consistent visual language. The intended accessibility design includes Dark Mode, Dynamic Type and responsive layouts. SwiftUI state-driven rendering helps the interface respond to task changes and user actions.

EVIDENCE TO ADD / VERIFY: Name the actual custom component/modifier and show code or screenshot evidence. Identify the animations, transitions, gestures and adaptive/orientation tests actually implemented. Add icon/branding evidence.

2.4 Navigation and architecture

StudyWell separates views, task models, persistence, service integration and view-model logic in an MVVM-style organisation. NavigationStack (or the navigation container present in the final source) supports movement between the dashboard, task list, task editor and wellness screens. This separation makes UI changes less likely to interfere with data-handling logic.

2.5 Data persistence

Core Data is the documented persistence approach for structured task records. It is intended to preserve task titles, categories, due dates and completion state between app launches. UserDefaults, accessed via @AppStorage where appropriate, is used for small preference values such as weekly and daily goals. These are different storage responsibilities: Core Data manages task records, while UserDefaults manages lightweight settings.

EVIDENCE TO ADD / VERIFY: Demonstrate creating a task, force-quitting/relaunching, and confirming persistence; record the result.

2.6 Advanced framework: HealthKit

HealthKit is the selected advanced iOS framework. It supports reading available daily step totals after the required authorisation, allowing students to compare their movement against a personal goal. The app should handle missing or inaccessible data without implying that the user denied permission. Health data is not described as uploaded to an external server.

EVIDENCE TO ADD / VERIFY: Confirm HealthKit entitlement, permission text, tested iPhone model, access outcome and screenshot. Simulator behaviour is not proof of access to real step data.

2.7 Notifications, privacy and limitations

UserNotifications is the documented approach for local deadline reminders, subject to the user granting notification permission. The current scope is individual task and wellness planning; cloud collaboration, account synchronisation and remote notifications are outside the documented core feature set.

3. Part A — User guide, testing and reflection

3.1 User guide

Launch StudyWell and review the Today dashboard.

Open Tasks and create a task with a title and deadline; add optional module, priority and notes.

Save the task, then edit it or mark it complete from the task list.

Use search or filters to locate specific tasks.

Open Goals & Wellness and configure weekly task and daily step targets.

If prompted, choose whether to allow HealthKit step access and deadline notifications.

Reopen the application to verify saved tasks and preferences remain available.

Note: the exact labels and interaction order should be updated to match the final build.

3.2 Testing methodology and results

The assignment specifically requests unit tests, UI tests, user acceptance testing and performance optimisation. The table below is a verification plan, not a claim that the tests have passed.

Test ID	Test / expected result	Observed result
A01	Create and reopen task: stored task remains after relaunch	[NOT RECORDED]
A02	Edit, complete, reopen and delete tasks: UI and data stay consistent	[NOT RECORDED]
A03	Search/filter: matching tasks displayed	[NOT RECORDED]
A04	Reminder permission and scheduled notification	[NOT RECORDED]
A05	HealthKit authorised/unavailable states handled	[NOT RECORDED]
A06	Dark Mode, large text and device/orientation layouts	[NOT RECORDED]
A07	XCTest unit tests and UI tests	[ATTACH XCODE RESULTS]
A08	Launch time, responsiveness and memory observations	[NOT MEASURED]
A09	User acceptance: participant, task, feedback	[NOT CONDUCTED / ADD]

3.3 Challenges, design decisions and reflection

Development was split between a Windows source-editing environment and a Mac used for Xcode builds. This workflow required Git-based synchronisation, careful file organisation and validation of target membership, capabilities and Apple-specific build settings. Separating persistent task data from preferences and external framework services was intended to keep the app maintainable.

Reflection: integrating wellness data into a productivity app creates a more holistic student experience, but it also introduces permission, device-availability and privacy considerations. Future improvements

could include richer calendar views, cross-device synchronisation and more detailed goal analytics. These are proposed enhancements, not implemented features.

4. Part B — QuizRoom (tvOS)

4.1 Innovative concept and target audience

QuizRoom is a local multiplayer trivia experience designed for Apple TV. It uses a shared large screen and one remote to support a social, turn-taking activity for friends, families and small groups. The concept emphasises accessibility of group play without account creation, internet connectivity or multiple controllers.

The prototype is positioned as a living-room group activity rather than a phone-first quiz transferred unchanged to television. Its tvOS-specific interaction includes focus-based answer selection, large-screen readability and clear player-turn feedback.

4.2 Existing similar applications and differentiation

The assignment requires descriptions of existing similar apps. Suggested comparison candidates are Kahoot! (hosted group quiz sessions) and Jackbox Party Pack titles (social television party games). Their current Apple TV availability, exact feature sets, prices and store listings have NOT been independently verified in this report. Verify official sources and cite them before submission.

Comparison aspect	QuizRoom	Similar-app research to complete
Participation	2–4 local players taking turns on one remote	[VERIFY Kahoot! and Jackbox interaction models]
Questions	Bundled categories and questions	[VERIFY competitors' content models]
Connection	Core game playable offline	[VERIFY competitors' connection needs]
Result	Winner and ranked leaderboard	[VERIFY competitor results features]

4.3 Core prototype features

Setup: enter player names, select 2–4 players and choose a category.

Question bank: documented 20 bundled questions across General Knowledge and Science & Technology.

Turn taking: each player receives two questions.

Answer feedback: selection locks, correct/incorrect colours and an explanation are shown.

Scoring: correct answer earns 10 points; wrong answer earns zero.

Results: winner, leaderboard and options to play again or start a new game.

Observed simulator evidence from the development conversation includes a question screen, correct-answer feedback, incorrect-answer feedback and a final leaderboard showing Jathu 20 points and Thivi 10 points. This supports the core manual gameplay flow; it does not by itself verify all edge cases or automated tests.

4.4 Platform interaction and interface design

The Apple TV interface prioritises large typography, contrast and remote-controlled focus. AnswerButton is a reusable SwiftUI component that handles answer labelling, focus presentation and correctness feedback. QuizTheme centralises colours and appearance. The prototype was run in the Apple TV 4K (3rd generation), tvOS 27.0 simulator.

4.5 Architecture and source organisation

Source file	Role
QuizRoomApp.swift	SwiftUI app entry point and shared QuizGame object
SetupView.swift	Player and category setup
GameView.swift	Questions, answer options and progress
ResultsView.swift	Leaderboard and replay actions
QuizGame.swift	Observable game phases, turns, scores and winner logic
QuizPlayer.swift / QuizQuestion.swift	Player and question data models
QuestionBank.swift	Locally bundled quiz content
AnswerButton.swift / QuizTheme.swift	Reusable focused answer UI and visual theme

The game state moves between setup, playing and results. SwiftUI observes the shared QuizGame object. Turn assignment can be expressed using the current question index modulo the number of players. The game blocks duplicate answer submission and calculates results after the final question.

4.6 Testing, constraints and future improvements

Test	Evidence / status
B01: Xcode build and simulator launch	Observed: app running on Apple TV 4K simulator
B02: Question and answer navigation	Observed: screenshots of gameplay
B03: Correct/incorrect feedback	Observed: screenshots of both states
B04: Results leaderboard	Observed: final results screenshot
B05: Ties, replay, reset, all player counts	[ADD TEST EVIDENCE]
B06: XCTest automated tests	[NOT VERIFIED AS RUN]
B07: Remote focus and accessibility	[ADD TEST EVIDENCE]

Current scope limitations include no online multiplayer, no cloud profiles, no persistent scores and a limited bundled question bank. Future versions could introduce larger question packs, accessibility options, localisation and optional online participation.

5. Part B — Stakeholder pitch and commercial potential

5.1 Value proposition

QuizRoom turns an Apple TV into a simple shared quiz experience that is immediately playable with a single remote. Its differentiator is frictionless local group play, with clear turns, feedback and a visible leaderboard.

5.2 Target market

Proposed primary segments: families and friends who already own an Apple TV, small social gatherings, and informal education or club settings. This is a hypothesis, not a measured market-size estimate.

5.3 Revenue model

Proposed freemium approach: a free starter pack of questions and optional paid themed question packs. A one-time premium unlock is another possibility. No monetisation is implemented in the current prototype; pricing and willingness to pay would need user research.

5.4 Go-to-market strategy

Validate the experience with small Apple TV user-testing sessions and collect usability feedback.

Expand the question bank and improve remote focus, accessibility and content variety.

Prepare App Store product screenshots and a short gameplay demonstration.

Launch to a narrow audience of Apple TV owners; gather ratings and retention feedback.

Evaluate optional paid content only after confirming sustained user interest.

5.5 Short viva pitch

"QuizRoom makes quiz night simple. Instead of asking every participant to install an app or create an account, it turns the Apple TV already in the living room into a shared trivia experience. Players take turns with one remote, get instant answer feedback and finish with a leaderboard. Our prototype demonstrates the complete local gameplay loop, while future question packs could support an affordable content-led business model."

6. Development workflow, version control and AI disclosure

6.1 Tools and workflow

Swift and SwiftUI were used for both native applications. Source files were authored and edited using Cursor/VS Code on Windows, synchronised with Git and GitHub, and integrated into Xcode projects on a Mac. Xcode supplied the Apple SDKs, build process, debugging tools and iOS/tvOS simulators. The tvOS app was manually exercised on the Apple TV simulator.

6.2 GitHub and reproducibility

The brief requires a separate repository for each project, meaningful commits, clear READMEs and build instructions. Each final repository should contain the Xcode project, app source, required assets, tests (if created), README and appropriately named evidence screenshots. Feature branches and pull requests should be described only if actually used.

EVIDENCE TO ADD / VERIFY: Insert StudyWell URL; confirm both repos include final Xcode project and source, build instructions, screenshots and final commits.

6.3 AI-assisted development disclosure

AI assistance was used during the development process for discussing assignment requirements, planning features and architecture, drafting or refining Swift/SwiftUI source, explaining errors, troubleshooting Xcode integration, and preparing documentation. AI-generated suggestions were subject to human review, editing and simulator validation where performed. This disclosure should be expanded with an actual prompt-and-response record as explicitly required by the assignment brief.

Example interaction topics evidenced by the development conversation: (1) structuring StudyWell using Core Data and HealthKit; (2) planning QuizRoom gameplay and tvOS screens; (3) resolving Xcode project and target issues; (4) improving the QuizRoom results interface; (5) explaining technologies for the viva.

EVIDENCE TO ADD / VERIFY: Attach actual prompt excerpts and responses or a dated exported interaction log. Do not present illustrative examples as verbatim transcripts.

6.4 Originality, third-party assets and references

The student must review the final code, verify that they understand its implementation, and declare all third-party code, imagery, audio, question sources, libraries and tutorials actually used. Apple frameworks (SwiftUI, Core Data, HealthKit, UserNotifications and XCTest) are first-party platform technologies. Add citations for any external questions, icons or comparison research.

7. Live demonstration and final checklist

7.1 Suggested Part A demo

Open StudyWell → navigate across at least three screens → create and edit a task → complete a task → show persisted data after relaunch → show goals and the HealthKit step section → explain permission handling and reminders → show tests or screenshots.

7.2 Suggested Part B demo

Launch QuizRoom on the Apple TV simulator → enter two player names → choose category → start game → demonstrate remote focus and answer feedback → complete questions → show winner and leaderboard → test replay/new game.

7.3 Pre-submission checklist

Fill student identity, submission date and both repository URLs.

Add actual screenshots for every claimed feature, with figure numbers and captions.

Verify the final source files and update any differences from this draft.

Run and record unit tests, UI tests and user acceptance tests where possible.

Confirm persistence, HealthKit permissions, notifications and accessibility behaviour.

Research and cite comparable tvOS applications using reliable current sources.

Attach AI prompt/response evidence and third-party attributions.

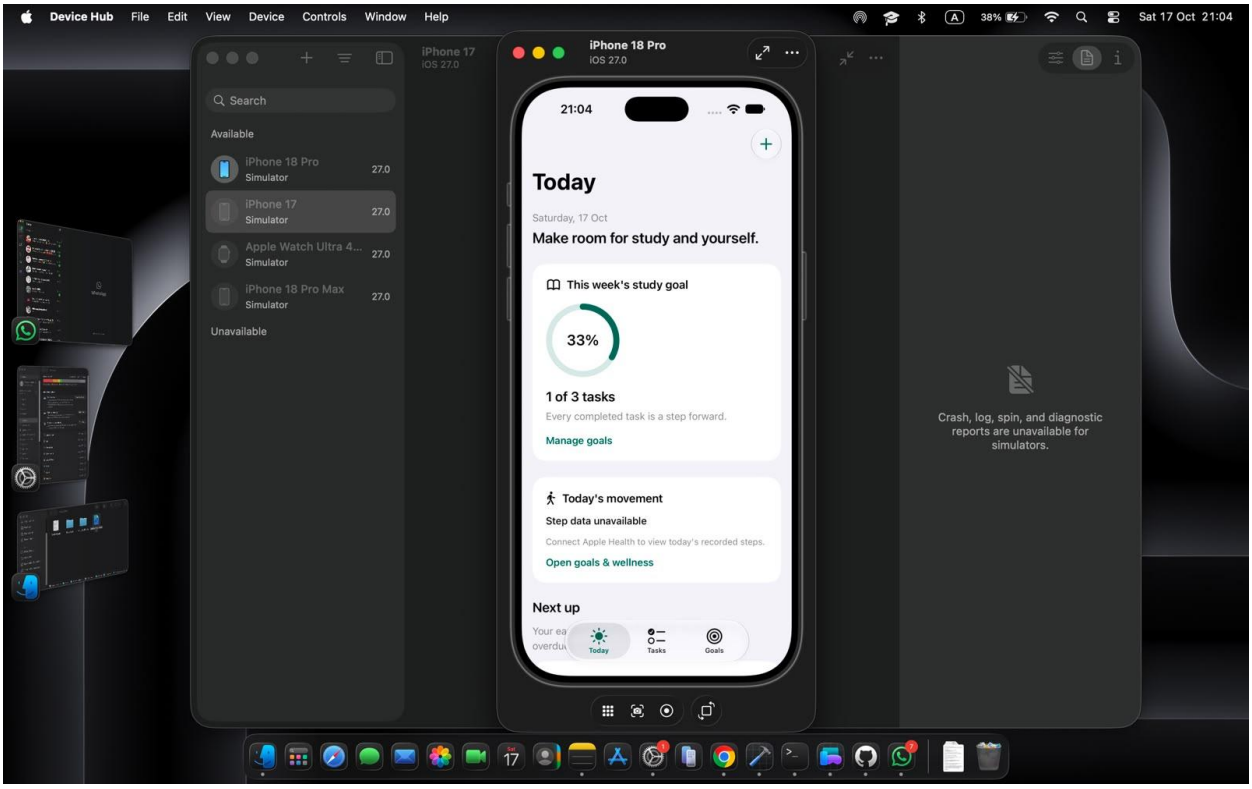
Confirm README build/run instructions and complete GitHub commits.

Prepare a separate pitch slide deck if the lecturer expects slides; this report does not replace the pitch presentation.

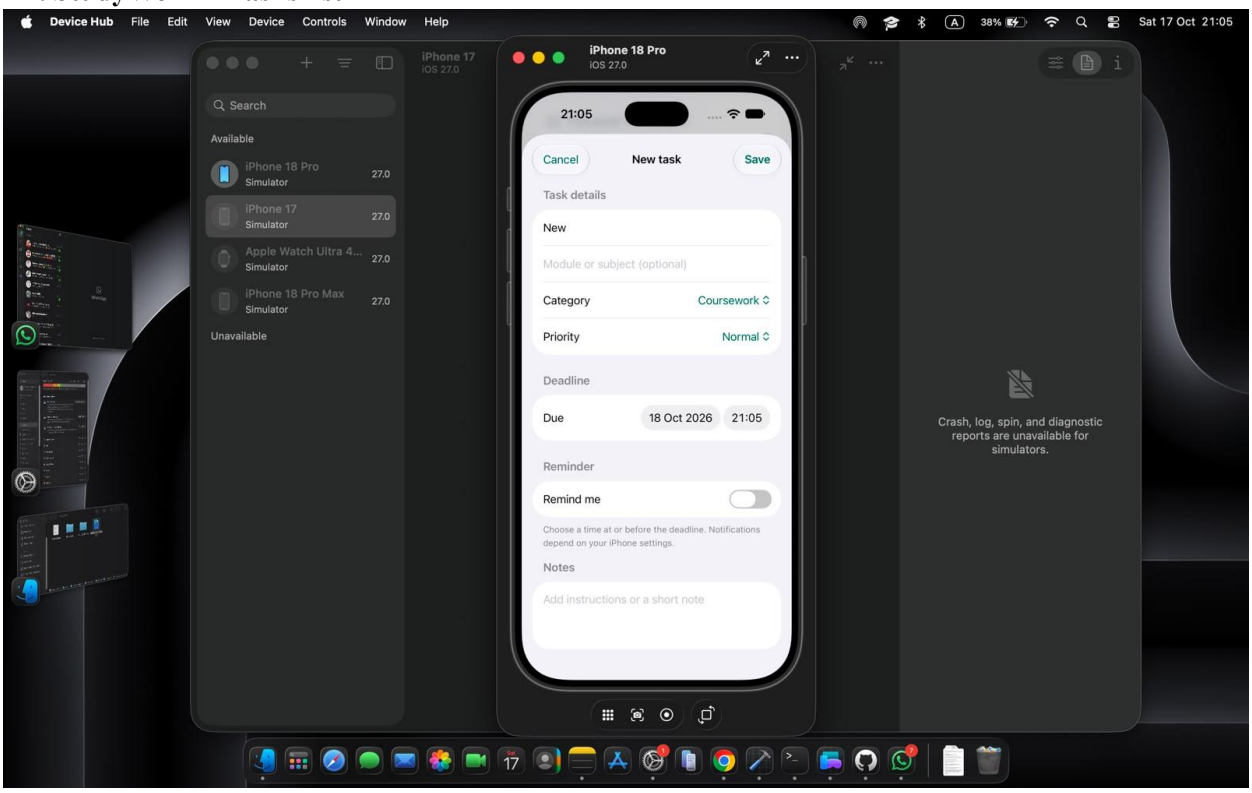
Rehearse the live demonstrations and verify submission instructions on the course portal.

Appendix A — Screenshot evidence placeholders

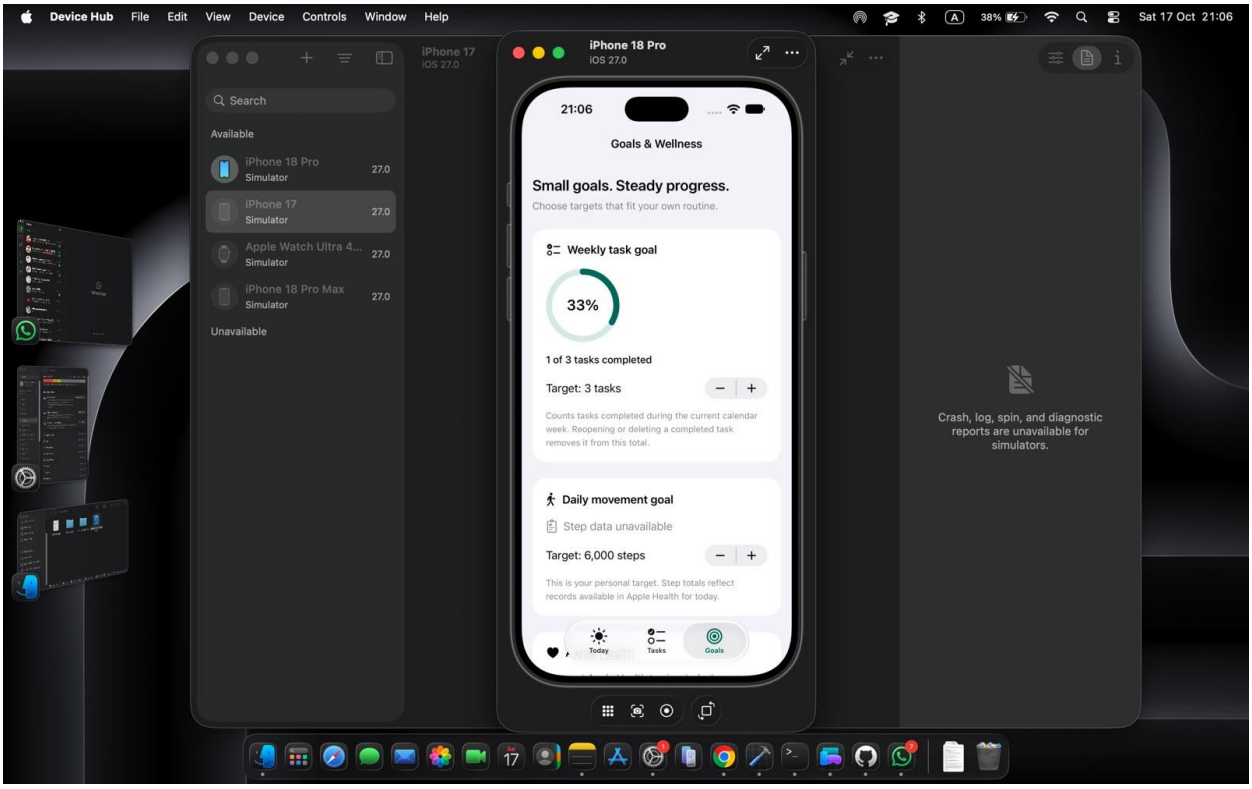
A1. StudyWell — Today dashboard



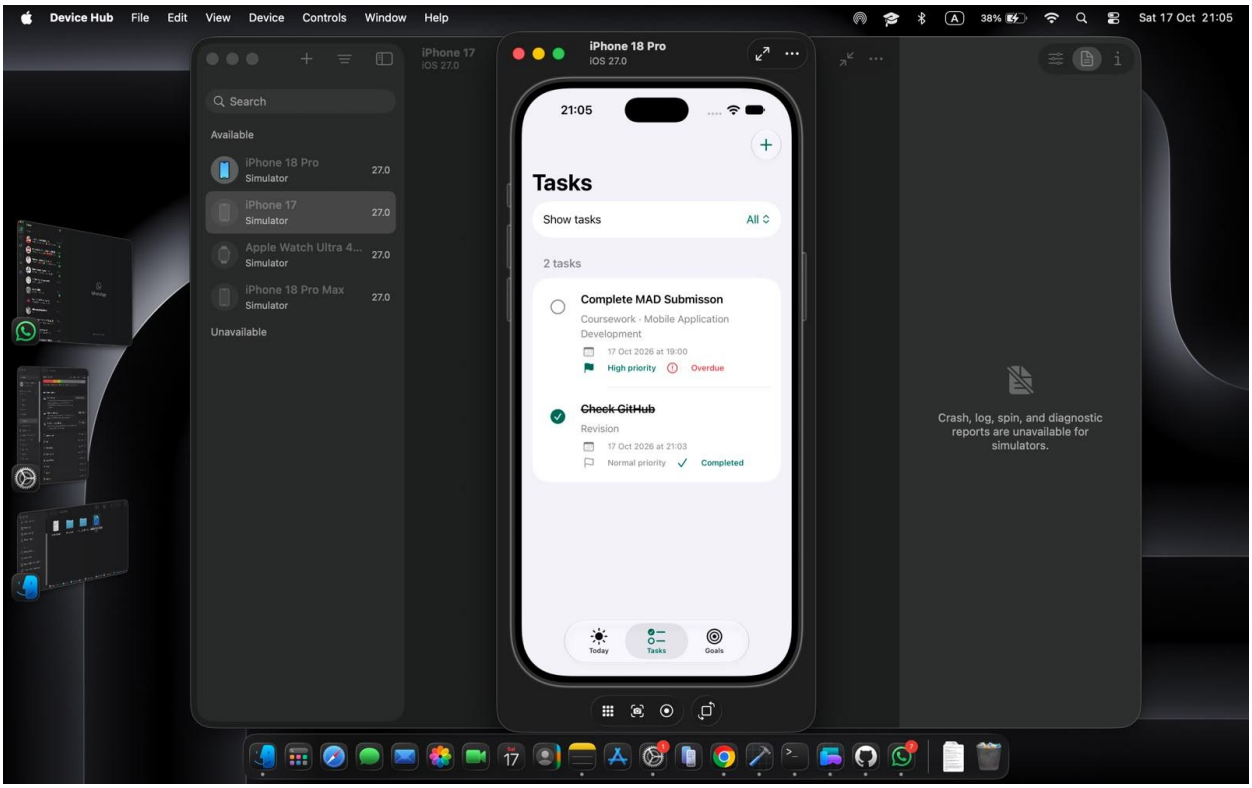
A2. StudyWell — Tasks list



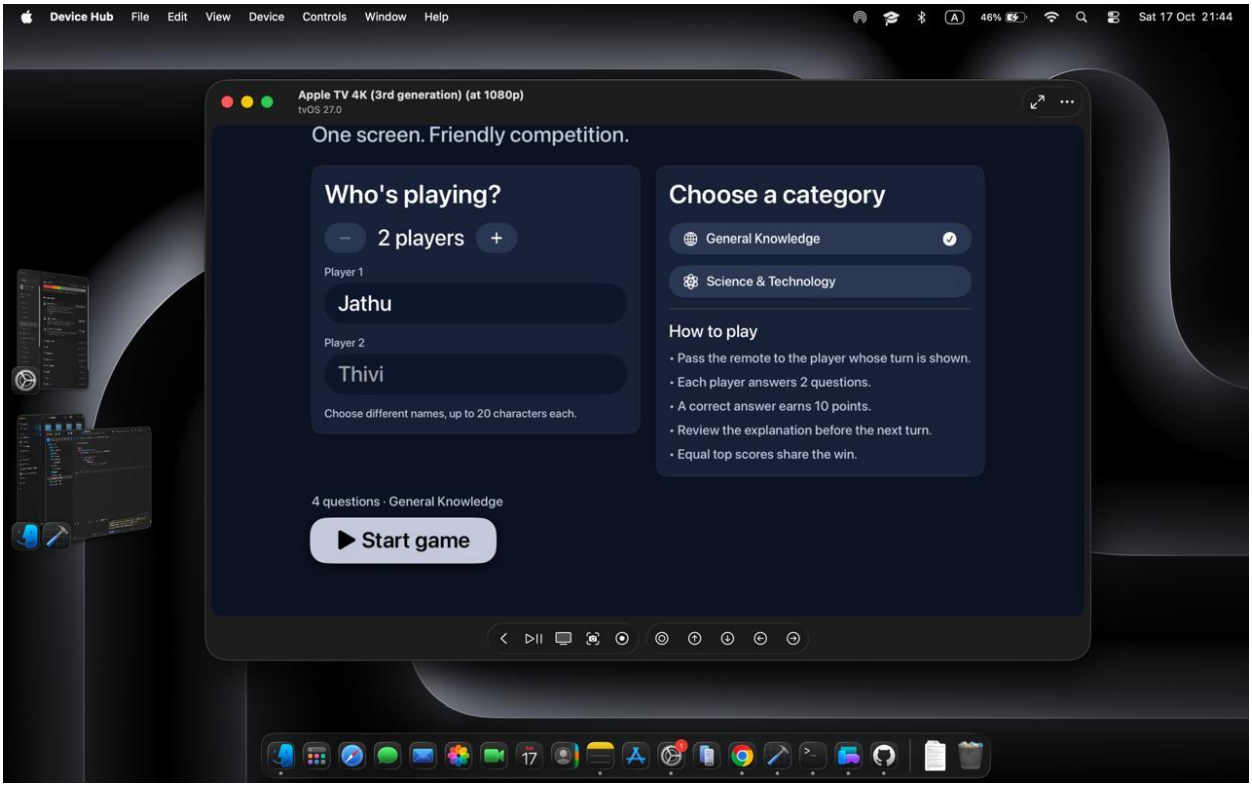
A3. StudyWell — Add/Edit Task



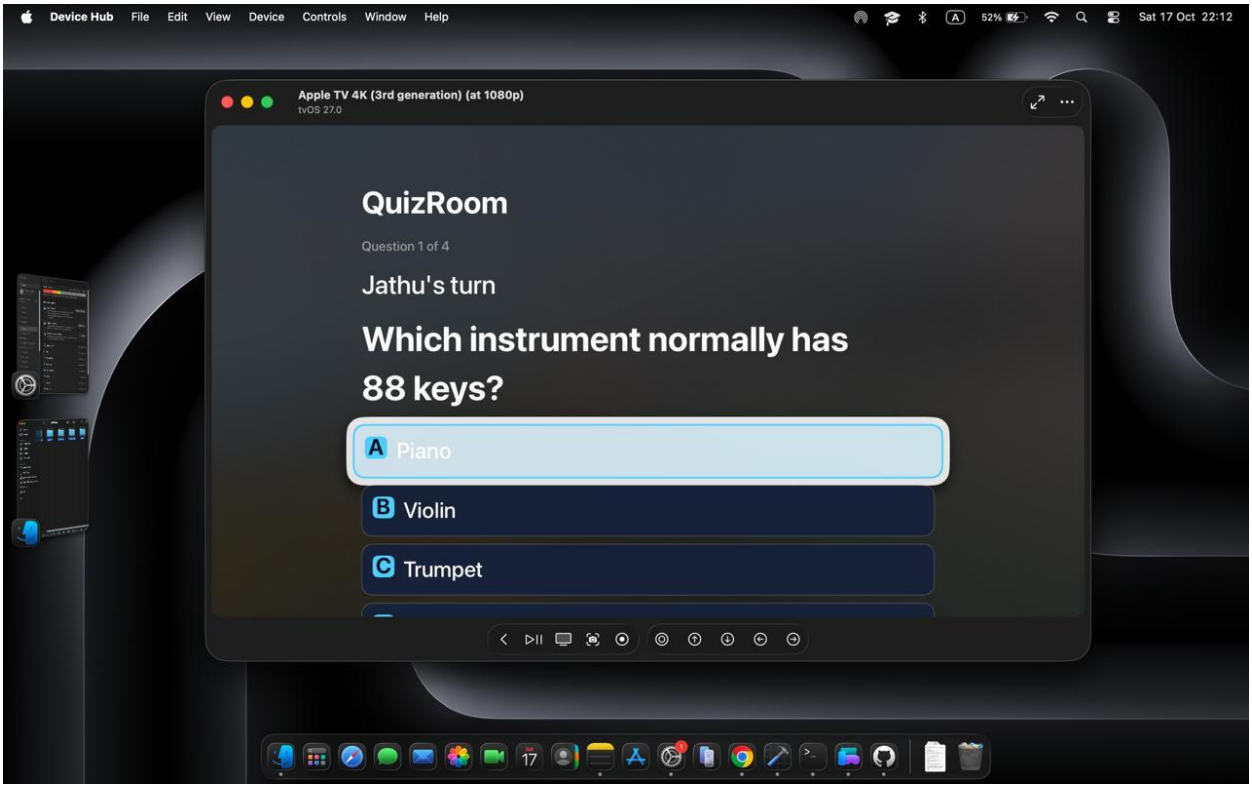
A4. StudyWell — Goals & Wellness / HealthKit



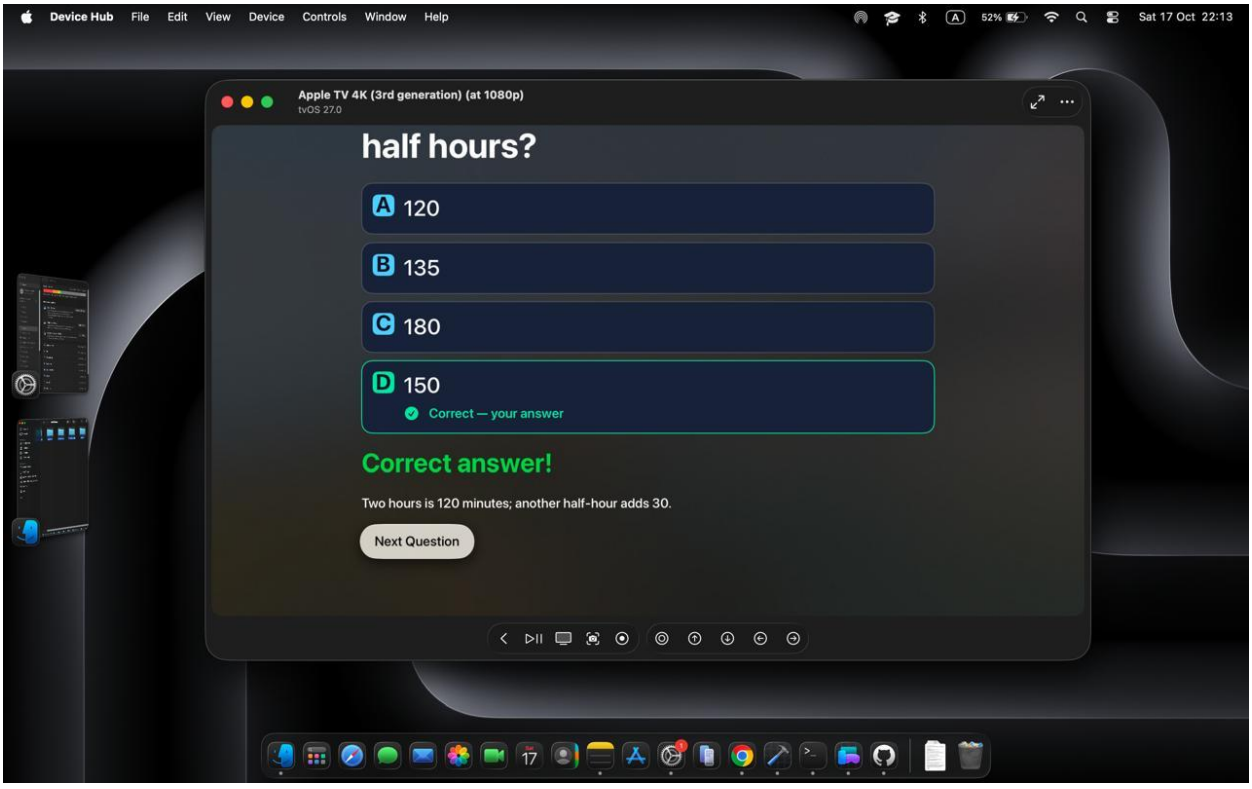
B1. QuizRoom — setup screen



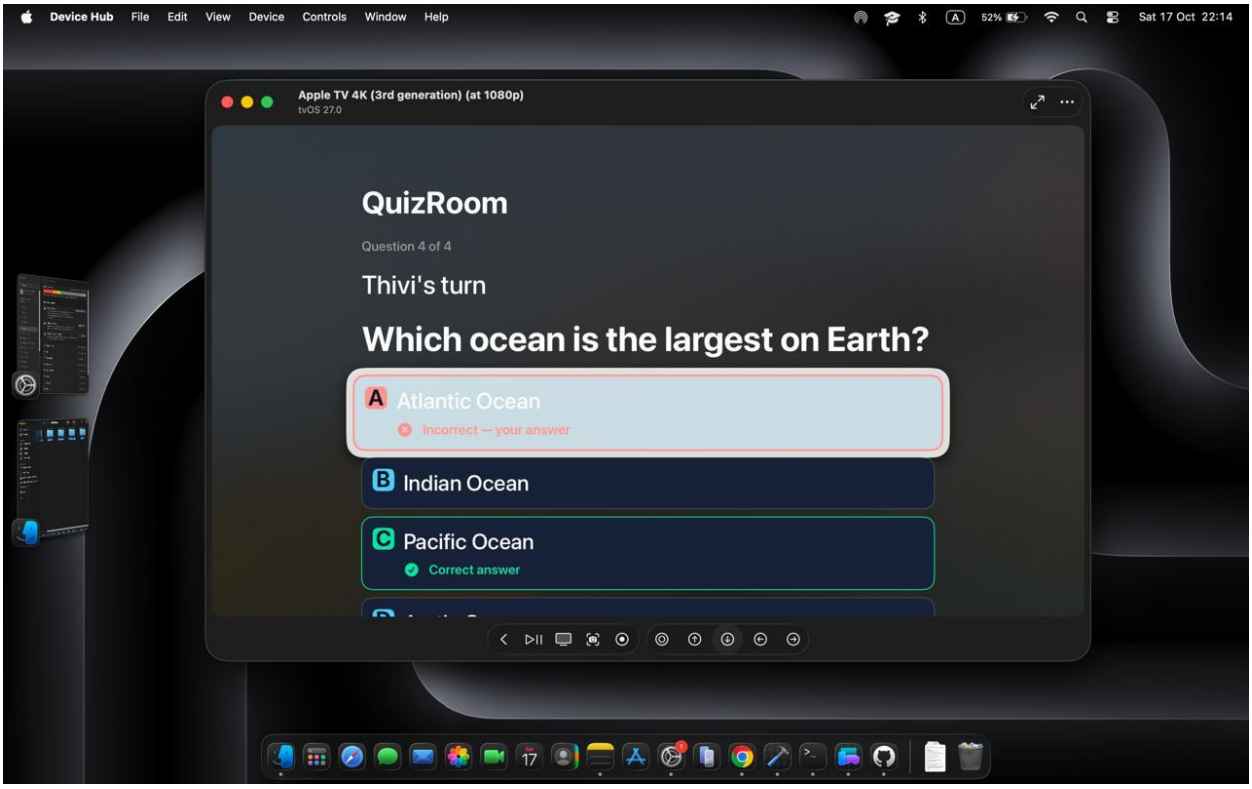
B2. QuizRoom — question screen



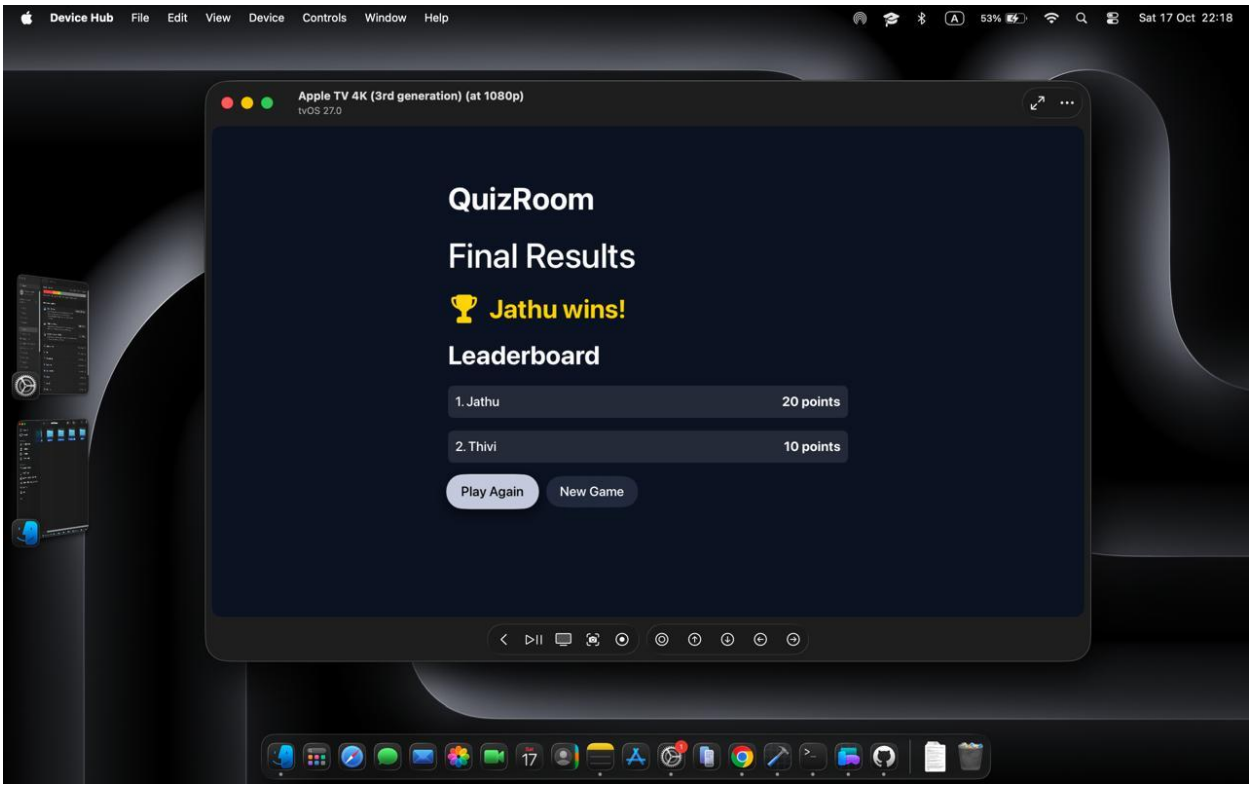
B3. QuizRoom — correct-answer feedback



B4. QuizRoom — incorrect-answer feedback



B5. QuizRoom — final leaderboardwh



Appendix B — AI interaction record template

Date / tool	Actual prompt (verbatim excerpt)	Response summary or excerpt	What I changed / verified
ChatGPT/Cursor	[PASTE PROMPT]	[PASTE RESPONSE]	[EXPLAIN VALIDATION]
ChatGPT/Cursor	[PASTE PROMPT]	[PASTE RESPONSE]	[EXPLAIN VALIDATION]

Appendix C — References

SE4041 Mobile Application Design and Development, Assignment 02 — Swift, Faculty of Computing, Semester 2, 2026 (user-supplied assignment brief).

Apple Developer Documentation — SwiftUI, Core Data, HealthKit, UserNotifications and tvOS Human Interface Guidelines [ADD ACCESSED DATES / EXACT URLS].

[ADD VERIFIED SOURCES FOR COMPARABLE APPS, QUESTION CONTENT AND THIRD-PARTY RESOURCES].